Dynamic Programming Supervised Learning in Intraday Trading

Robert Mertens[1] and **Finn van Oosterhout**[2]
[1]First Affiliation
[2]Second Affiliation
mertensrh@gmail.com, finn.oosterhoutje@gmail.com

Abstract

We introduce Dynamic Programming Supervised Learning (DPSL), a novel framework that transforms the task of training a reinforcement learning agent in a deterministic intraday trading environment into a stable and efficient supervised learning (SL) problem. The loss function of the SL agent relies on an approximation of the optimal value function that can be computed in polynomial time using dynamic programming (DP). We demonstrate that DPSL converges significantly faster and more reliably than traditional RL, even in the absence of strong predictive signals.

1 Introduction

TODO: missing

2 Background

TODO: missing

3 Problem Formulation

3.1 Prices

Timeframes

The trading simulation is divided into a finite T timeframes, indexed by $t \in \{0, 1, \ldots, T-1\}$. For each timeframe, we are given the Open, High, Low, Close (OHLC) bid/ask prices. To denote the prices we use a subscript for the timeframe t, a second subscript for the price type (o, h, l, c), and a superscript for the side of the book (b for bid, a for ask). For example, the closing bid price at t is denoted by $P^b_{t,c}$.

Orders

At the end of each timeframe $t-1$, the agent decides some shares to trade ΔS_t. This order will then be executed somewhere during the start of timeframe t. We assume zero-market impact, which means the price at which the order is executed is independent of the absolute order size $|\Delta S_t|$. Critically, the agent may only base its decision on historical information, including but not limited to past prices and account states.

Slippage

In real trading scenarios the agent will encounter delays. We define this delay as the time between the end of timeframe $t-1$ (decision time), and the time at which the corresponding order ΔS_t would be executed (execution time). During this delay the price may also change. We say the decision price $P_{t-1,c}$ is the price the trade would be executed at if there was no delay, and the execution price P_t is the actual price the trade would be executed at. Then slippage may be defined as the difference between the execution and the decision price.

The decision price $P_{t-1,c}$ is known when deciding on ΔS_t, the execution price P_t is not. The execution price may be estimated through various methods. For this paper, the execution price is derived by assuming a constant delay and retrieving the execution price from tick data. Alternative methods include using the opening price $P_{t,o}$ or assuming worst case slippage.

3.2 Account state

C_t, and S_t denote the cash and shares of the account between the execution of ΔS_t and ΔS_{t+1}. Using the account state and prices, we can determine state-price derivatives as described in table 1.

Table 1: State Price Derivatives

Term	Formula
Valuation price	$P^*_t = \begin{cases} P^{\mathrm{b}}_t & S_t \geq 0 \\ P^{\mathrm{a}}_t & \text{otherwise} \end{cases}$
Positional value	$p_t = S_t \cdot P^*_t$
Equity	$e_t = C_t + p_t$
Exposure	$\epsilon_t = p_t / e_t$

X'_t denotes some state-price derivative X relative to the next state C_{t+1}, S_{t+1} instead of the current state C_t, S_t.

Initial capital is cash only $S_0 = 0$, $C_0 = e_0$. For each step, the updated shares is given by $S_t = S_{t-1} + \Delta S_t$ and the updated cash is given by $C_t = C_{t-1} - \Delta S_t \cdot \varphi_t$ where φ_t are the costs per share traded taking commission percentage

$k \in [0, 1]$ and bid-ask spread into account.

$$\varphi_t = \begin{cases} (1+k) \cdot P_t^{\text{ask}} & \text{if } \Delta S_t \geq 0 \\ (1-k) \cdot P_t^{\text{bid}} & \text{otherwise} \end{cases}$$

Goal

The goal is to develop an agent capable of deciding on a sequence orders $\Delta S_1, \Delta S_2, \ldots, \Delta S_{T-1}$, that maximizes the expected final equity $e_{T-1,c}$ given some arbitrary initial equity e_0, on unseen data.

4 MDP Derivation

We can formulate the trading problem as a Markov Decision Process (MDP). An MDP is formally defined by a 5-tuple $(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P}, \gamma)$, representing the state space, action space, reward function, transition dynamics, and discount factor, respectively. The agent interacts with the environment (the market) in discrete time steps, which correspond to the timeframes t of our problem.

4.1 Goal equivalence

The MDP must be formulated appropriately such that the agents goal is the exactly the goal described in 3.2. For this purpose the action space $\mathcal{A}$ and reward function $\mathcal{R}$ must possess the following two properties.

Equity independence The agent's learned policy should be applicable regardless of the initial or current account equity. This means that for every action $a_t \in \mathcal{A}$, the corresponding reward r_t must be the same regardless of equity. Whether this property is met depends on the combination of action space and reward function.

Goal alignment The reward function must incentivize the maximization of the final equity $e_{T-1,c}$. This means that to maximize the total return $G_0 = \sum_{t=0}^{T-1} r_t$, should be equivalent to maximizing the final equity. To prove this is the case we need to show that for any two policies π_A and π_B, that π_A has a larger return than π_B, if and only if π_A has a larger final equity than π_B.

$$G_0^A > G_0^B \iff e_{T-1,c}^A > e_{T-1,c}^B$$

4.2 Action space

Target positions

The agent's output, a continuous value $a_t \in \mathcal{A}$, must be able to be translated into a concrete trading order ΔS_t. For effective learning, particularly with neural networks, it is standard practice for this output to be normalized, typically within a bounded space such as $\mathcal{A} = [-1, 1]$. The core design choice, therefore, lies in what this normalized value represents.

A fundamental distinction can be made between action spaces that define a change in position (buy, sell, hold) and those that define a target position (long, short, cash). While simple, defining an action as the change in position, or the shares to trade ΔS_t directly, is inefficient. Such a formulation burdens the agent with calculating how many shares are needed to close one position and open another. This is a simple arithmetic task that can be offloaded from the agent, allowing it to focus on the decision of what this target position should be. For this reason we focus on action spaces that define some target position.

Target shares

The most basic target position is the target number of shares S_t. Here, an action $a_t \in [-1, 1]$ can be mapped to a target number of shares, where 1 represents a maximum long position and -1 represents a maximum short position. However, this approach is not normalized with respect to the asset's price. If the price of the asset were to increase significantly, the same number of shares would represent a much larger capital risk, and vice versa.

Target positional value

To resolve this, we can define the action as a factor of the target positional value, $p'_{t-1,c} = S_t \cdot P'^{*}_{t-1,c}$. The maximum and minimum actions now correspond to fixed cash values, providing a more stable risk definition. Now price is relevant for the action a_t to order ΔS_t derivation. This is where the difference between decision price $P_{t-1,c}$ and execution price P_t becomes significant. Also, a change in price may cause the account's positional value to exceed the bounds of the action space, causing "holding" shares to not be a possible action in some cases.

Target exposure

For target positional values, the bounds are typically defined as some fixed value. This comes with the side-effect that as equity grows, a smaller percentage of equity will be invested and vice-versa. Suppose then that we use the agent's current equity as a dynamic bound on the maximal positional value. This directly leads us to the target exposure $\epsilon'_{t-1,c} = p'_{t-1,c}/e'_{t-1,c}$. Here, the agent's action $a_t \in [-1, 1]$ directly represents the fraction of its equity to be allocated to the position. This provides natural risk management as the agent will invest more as equity grows, and less as equity decreases.

4.3 Reward function

Profit and Loss

The most obvious reward measure is the change in equity, or Profit and Loss (PnL), defined as $r_t = e_{t,c} - e_{t-1,c}$. Trivially this reward is goal-aligned, as the total return G_0^{PnL} telescopes to the total PnL of the episode:

$$G_0^{\text{PnL}} = \sum_{t=1}^{T-1} (e_{t,c} - e_{t-1,c}) = e_{T-1,c} - e_{0,c}$$

Given a fixed initial equity $e_{0,c}$, maximizing the total return is mathematically equivalent to the maximizing the final equity $e_{T-1,c}$. When paired with an absolute action space like the target position, the action and reward are also equity-independent, as any action will generate the same PnL regardless of the account's total equity.

Return on Equity

If we were to use target exposures as our action space however, then the absolute PnL scales with equity, violating equity-independence. In an attempt to solve this, we can normalize the reward by the equity aswell, directly leading us to the Return on Equity (ROE):

$$r_t = \frac{e_{t,c} - e_{t-1,c}}{e_{t-1,c}}$$

However, for ROE rewards, maximizing the total sum of rewards (return) is not strictly aligned with maximizing the final equity, violating goal-alignment. A formal proof for this is provided in section A.

Log-equity ratio

To find a reward function that is both goal-aligned and equity-independent when using target exposures, we must start from the definition of the final equity itself.

$$e_{T-1,c} = e_{0,c} \cdot \prod_{t=1}^{T-1} \left(\frac{e_{t,c}}{e_{t-1,c}} \right)$$

Since the logarithm is a strictly monotonic function, maximizing $e_{T-1,c}$ is equivalent to maximizing $\log(e_{T-1,c})$. Applying the logarithm transforms the product into a sum:

$$\log(e_{T-1,c}) = \log(e_{0,c}) + \sum_{t=1}^{T-1} \log \left(\frac{e_{t,c}}{e_{t-1,c}} \right)$$

We can recognize the right hand side as the total return using some reward function r_t, namely the log-equity ratio:

$$r_t = \log \left(\frac{e_{t,c}}{e_{t-1,c}} \right) = \log(e_{t,c}) - \log(e_{t-1,c})$$

The agent's objective, maximizing the sum of these rewards, then becomes directly equivalent to maximizing the log of the final-to-initial equity ratio. Which given that the initial equity $e_{0,c}$ is fixed, is equivalent to maximizing the final equity $e_{T-1,c}$, showing goal-equivalence.

Pairing target exposure actions with a log-equity ratio reward function also meets the equity independence criterion. The target exposure ϵ_t ensures the absolute size of the position taken is proportional to the agent's equity. Consequently, the resulting Profit and Loss (PnL) from that position will also be proportional to the equity. The reward, $r_t = log(e_{t,c}/e_{t-1,c})$, is the logarithm of the single-period return. This return is equal to $1 + \text{PnL}_t/e_{t-1,c}$. Since both the PnL and the equity in the denominator scale linearly with equity, the equity term cancels out, making the return ratio independent of the absolute equity value, demonstrating equity-independence.

4.4 Additive vs Multiplicative

Given these derivations, we get the option to choose between an additive, or a multiplicative approach. As shown, both approaches provide goal-alignment and equity-independence, the question remains which properties are desired.

- **Additive approach**: pairing an absolute action space like the target positional value, with an absolute reward like PnL. This allows for constant risk taking.
- **Multiplicative approach**: pairing an action space that is relative to equity, like target exposure, with a relative reward, like the log-equity ratio. This adapts risk to capital, and allows for compound interest.

Up until now we have implicitly assumed that both the equity and prices remain strictly positive. We consider the consequences of relaxing these constraints in section C. Negative prices turn out to be easy to deal with, but multiplicative approaches struggle with negative equities.

Another thing to consider is the impact of changes in price on the position. Simply said, the ratio between the target position (positional value or exposure) decided upon at the end of timeframe $t-1$, and the resulting position at the closing price of timeframe t. We determine the maximal position-drift in section E.

4.5 Final MDP Specification

The state $s_t \in \mathcal{S}$ at the end of timeframe t encapsulates the complete history of all prices and account states up to that point. The policy of a Reinforcement Learning agent $\pi(a_t|o_t)$ would operate on an observation o_t, which is a feature vector (state representation) that the agent derives from the full state s_t.

Crucially, the state transitions $\mathcal{P}$ are entirely deterministic. Account states are determined using the formulae defined in section 3, and prices simply follow a fixed series of historical market prices which are not impacted by agent trades (following the zero market impact assumption). Consequently, for any given state s_t and action a_t the next state s_{t+1} can be derived deterministically.

Since the problem only focuses on the maximization of the final equity $e_{T-1,c}$, the discount factor γ shall be equal to one $\gamma = 1$.

In order to get negative equity using a multiplicative approach, the price must change by more than 100% in a single timeframe, given a 1:1 leverage cap. Since market prices used for this paper do not contain negative prices and are not particularly volatile, we may safely adopt the the multiplicative approach.

This means $a_t \in [-1, 1]$ shall directly represent the target exposure $\epsilon'_{t-1,c}$, providing natural risk management, and a 1:1 leverage ratio. The agent receives log-equity rewards $r_t = \log(e_{t,c}/e_{t-1,c})$. We enforce a no-bankruptcy constraint, terminating the episode if equity becomes negative and returning a reward of negative infinity. But given the aforementioned reasons, this constraint likely never needs to be enforced.

5 DP-optimal and Supervised Learning

5.1 Definition of Optimal

The optimal policy, denoted as π^*, is the policy that maximizes the total return G_0. Given the fact that the environment is deterministic, the optimal policy is a sequence of actions that is guaranteed to result in the highest possible final equity. We can define this optimal behavior using the Bellman equations, which recursively relate the value of a state to the values of subsequent states.

Given that the optimal action for a timestep t is equity-independent, then the optimal action relies only on the current timestep t (since it determines the current and future prices), and the decision time exposure $\epsilon_{t-1,c}$. **TODO: Explain why**

Using this reduction we can define the optimal policy by working backward in time. Let $r_t(\epsilon, a)$ be the immediate reward for executing action $a \in \mathcal{A}$ in state (t, ϵ), where ϵ is

some decision-time exposure $\epsilon_{t-1,c}$. Then the optimal action-value function $Q_t^*(\epsilon, a)$, is the return for taking action a in state (t, ϵ) and following the optimal policy thereafter.

$$Q_t^*(\epsilon, a) = r_t(\epsilon, a) + \gamma \cdot V_{t+1}^*(\epsilon')$$

Here ϵ' is the next decision-time exposure $\epsilon_{t,c}$ corresponding to the execution of $a \in \mathcal{A}$ in state (t, ϵ). The optimal state-value function V_t^* is the maximum return achievable from (t, ϵ).

$$V_t^*(\epsilon) = \max_{a \in A} Q_t^*(\epsilon, a) \quad V_T^*(\epsilon) = 0$$

Finally the optimal policy is to choose the action that maximizes the Q-function at each step:

$$\pi_t^*(\epsilon) = \arg\max_{a \in A} Q_t^*(\epsilon, a)$$

5.2 Computation using Dynamic Programming

Dynamic programming (DP) is a method for solving complex problems by breaking them down into simpler overlapping subproblems and storing the solutions to avoid recomputation. Following our definition of the trading problem, we can use dynamic programming to approximate the optimal policy and value-functions.

In order to compute the optimal policy and value functions we must be able to compute the immediate reward $r_t(\epsilon, a)$ and the next exposure ϵ' for any given state (t, ϵ) and action a. The challenge is that we don't have access to the absolute cash and shares. Luckily, since the action-reward pairing is equity independent, we can simulate the trade by assuming a normalized pre-trade equity $e_{t-1,c} = 1$. From this, we derive the corresponding pre-trade positional value $p_{t-1,c} = \epsilon$, cash $C_{t-1} = 1 - \epsilon$, and shares $S_{t-1} = p_{t-1,c}/P_{t-1,c}^*$. The valuation price $P_{t-1,c}^*$ depends on the sign of ϵ.

$$P_{t-1,c}^* = \begin{cases} P_{t-1,c}^b & \epsilon \geq 0 \\ P_{t-1,c}^a & \text{otherwise} \end{cases}$$

Now we have S_{t-1}, and C_{t-1} we can simply use the trading logic defined in section 3 or a derivation thereof to determine the resulting equity $e_{t,c}$ and exposure $\epsilon_{t,c} = \epsilon'$. Since we used a normalized pre-trade equity, the resulting reward simplifies nicely $r_t = \log(e_{t,c}/e_{t-1,c}) = \log(e_{t,c})$.

Given some sequence of actions $a_1, a_2, \ldots, a_{T-1}$ we can already derive the value functions. The issue remains how to find the *optimal* sequence of actions. First we must discretize the action space if it is continuous. A simple and intuitive method is then to just go over all sequences of actions $a_t, a_{t+1}, \ldots, a_{T-1}$ for every state (ϵ, t) and pick the sequence of actions that leads to the maximal return. But this is clearly not computationally feasible.

This is where dynamic programming comes in. We want to use the recursive nature of the optimal value functions to our advantage. The core challenge is that the exposure ϵ is continuous. To make the problem tractable, we must discretize both the state and action spaces. We create a grid of N_ϵ exposures $\mathcal{E}^{\text{DP}} = \{\epsilon_0, \ldots, \epsilon_{N_\epsilon - 1}\}$ and N_a actions $\mathcal{A}^{\text{DP}} = \{a_0, \ldots, a_{N_a - 1}\}$.

Then we can approximate the optimal value function $V^{\text{DP}}(t, \epsilon_i)$ in our grid using backward induction.

1. **Terminal**: Any state after the final state $t = T - 1$ is valued equally. So their value is zero for all exposures $V_T^{\text{DP}}(\epsilon_i) = 0$ for all $\epsilon_i \in \mathcal{E}^{\text{DP}}$.
2. **Backwards induction**: We work backward from $t = T - 1$ down to $t = 0$. For each timestep t and each discrete exposure state $\epsilon_i \in \mathcal{E}^{\text{DP}}$, we calculate the Q-value for every possible discrete action $a_j \in \mathcal{A}^{\text{DP}}$.

$$Q_t^{\text{DP}}(\epsilon_i, a_j) = r_t(\epsilon_i, a_j) + V_{t+1}^{\text{DP}}(\epsilon_i')$$

3. **Interpolate**: The resulting next-state exposure ϵ_i' from taking action a_j will likely not fall exactly on one of our grid points $\epsilon_k \in \mathcal{E}^{\text{DP}}$. To find its value, $V_{t+1}^{\text{DP}}(\epsilon_i')$, we must interpolate using the known values at the neighboring grid points we've already computed for timestep $t + 1$ (e.g., using linear interpolation).
4. **Select maximum**: After computing the Q-value for all possible actions, the optimal value for the state (t, ϵ_i) is the maximum among them:

$$V_t^{\text{DP}}(\epsilon_i) = \max_{a_j \in \mathcal{A}^{\text{DP}}} Q_t^*(\epsilon_i, a_j)$$

5. **Memoize**: We store this value $V_t^{\text{DP}}(\epsilon_i)$ and the corresponding best action.

$$\pi_t^{\text{DP}}(\epsilon_i) = \arg\max_{a_j \in \mathcal{A}^{\text{DP}}} Q_t^{\text{DP}}(\epsilon_i, a_j)$$

By repeating this process until $t = 0$, we populate a complete table of optimal values and actions for every point in our discretized state-time grid. This reduces the problem's complexity from exponential to polynomial (approximately $O(T \cdot N_\epsilon \cdot N_a)$), making it computationally feasible. The trade-off for this efficiency is that the resulting policy is an approximation of the true optimal, with the accuracy depending on the granularity of the state-action discretization and the precision of the interpolation method.

5.3 Determining the difference from true optimal

TODO: missing

5.4 Supervised Learning

Given our close approximation of the optimal value function, V^{DP}, and its corresponding action-value function, Q^{DP}, we can re-frame the reinforcement learning task as a supervised learning problem. The goal is to train a parameterized policy $\pi_\theta(o_t)$, that learns to replicate the optimal decision-making process derived from our dynamic programming solution.

We define the loss for taking an action a in a state defined by (t, ϵ) as the difference between the value of the optimal action and the value of the action taken:

$$\mathcal{L}_t(\epsilon, a) = V_t^{\text{DP}}(\epsilon) - Q_t^{\text{DP}}(\epsilon, a)$$

By definition, this loss function has the desirable property that it is zero for the optimal action $a^* = \pi_t^{\text{DP}}(\epsilon)$ and strictly positive for any suboptimal action. Minimizing this loss is therefore equivalent to finding the optimal action.

The agent π_θ takes the same observation o_t as may be provided to the RL-agent and outputs an action $a_t = \pi_\theta(o_t)$. The

training objective is to adjust the parameters θ to minimize the loss produced by its chosen action. This objective directly pushes the policy network to output actions that maximize the Q-value for any given state, thus learning the optimal policy π^{DP}. Since π^{DP} was constructed to maximize the final equity, this supervised objective is perfectly aligned with the original reinforcement learning goal.

This approach draws strong parallels with actor-critic methods. The pre-computed functions V^{DP} and Q^{DP} serve as a perfect, static oracle critic. The term $Q_t^{\text{DP}}(\epsilon, a) - V_t^{\text{DP}}(\epsilon)$ is the advantage of action a, and our loss is simply the negative of this advantage. In traditional actor-critic learning, the critic is learned alongside the actor through environmental interaction, which can be unstable and sample-inefficient. Here, the actor (π_θ) learns from a fixed, optimal critic, turning the policy improvement step into a deterministic supervised learning problem. This sidesteps the need for exploration and temporal difference learning, leading to a much more stable and direct training process.

5.5 Risk for overfitting

Supervised Learning in a Nutshell

We are confronted by a possibly infinite series of inputs. The goal is to find a function that maps these inputs to the correct outputs, minimizing some loss function. The problem is, we only have access to the outputs for a subset of the inputs. Therefore we must find a pattern in these inputs that generalizes to unseen data.

The pattern we seek to identify is the signal, which represents the true relationship between (part of) the inputs and (part of) the output. Both the inputs and output may contain noise. For the input, its information that does not have any true relationship with the output, and for the output its information that cannot be predicted using the input. Noise includes measurement errors, and random chance. Output noise can also be a consequence of missing inputs.

Underfitting is when a model is too simple to learn the signal. Overfitting is when a model starts to learn the noise. Either because it finds spurious connections between the noise in the input and the output, or because it contorts itself to perfectly match the irreducible noise present in output. The key problem is to balance underfitting and overfitting. To learn as much of the signal as possible, without accidentally learning the noise.

Application to DPSL

TODO: missing

6 Methodology

TODO: missing

6.1 DPSL: Impact of shuffling

TODO: missing

6.2 DPSL: Actuality weighting

TODO: missing

6.3 Usage of SdAEs

TODO: missing

6.4 Exploration vs Exploitation

TODO: missing

7 Experimental setup

The ideal experiment should provide an example of a realistic and fair scenario in which the performance of RL vs DPSL can be isolated and compared.

7.1 Problem and Statics

TODO: missing

7.2 RL setup

TODO: missing

7.3 DPSL setup

TODO: missing

8 Results

TODO: missing

9 Discussion

9.1 Generalizability

One concern might lie in the generalizability of DPSL. After all, quite a few assumptions were made during the problem formulation that allowed the optimal value functions to be derived using dynamic programming. Who is to say these results generalize to real trading scenarios, or other markets?

The strongest assumptions that were made are that trade volume does not influence the price at which current and future trades are executed (zero market impact), and that we assume only reward functions are used that are independent of equity.

9.2 Comparison Realism

TODO: missing

9.3 Reproducibility

10 Conclusion

TODO: missing

References

A ROE goal-alignment counter-example

We show maximizing the sum of ROE is not the same as maximizing the final equity by providing a counter example.

Suppose we have two policies π^A and π^B. π^A follows a high volatility strategy experiencing a large loss of -50% followed by a large gain of 90%. Policy π^B follows a lower volatility strategy experiencing a smaller loss of -10% followed by a gain of 10%. The total return G_0^{ROE} and the final equity $e_{T-1,c}$ for both policies are then compared in table 2.

As shown in the table, π^A has a higher return ($G_0^A > G_0^B$), but π^B results in a higher final equity ($e_{T-1,c}^B > e_{T-1,c}^A$). This directly contradicts the condition for goal-alignment ($G_0^A > G_0^B \iff e_{T-1,c}^A > e_{T-1,c}^B$). The discrepancy

	G_0^{ROE}	$e_{T-1,c}$
π^A	$-0.5 + 0.9 = 0.4$	\$95
π^B	$-0.1 + 0.1 = 0.0$	\$99
Winner	π^A	π^B

Table 2: ROE Goal-Misalignment Counterexample

arises because the sum of simple returns (ROE) does not correctly account for the compounding nature of equity growth. A large percentage gain on a diminished capital base, as seen in Policy A, can misleadingly inflate the total return.

B Action to order derivation

B.1 Target Positional Value

The shares to trade ΔS_t is derived such that, given a decision price $P_{t-1,c}$ and a commission percentage $k \in [0, 1]$, the positional value after the trade $p'_{t-1,c}$ (if it were to be executed without delay at the decision price) is exactly the target positional value a_t times some normalization factor N.

TODO: Provide this derivation

B.2 Target Exposure

The shares to trade ΔS_t is derived such that, given a decision price $P_{t-1,c}$ and a commission percentage $k \in [0, 1]$, the exposure after the trade $\epsilon'_{t-1,c}$ (if it were to be executed without delay at the decision price) is exactly the target exposure a_t.

Given $a_t = \epsilon'_{t-1,c}$ we solve for ΔS_t. First, we try to define a_t in known terms. Trivially, $S_t = S_{t-1} + \Delta S_t$, so the post-trade positional value can be determined by $p'_{t-1,c} = S_t \cdot P'^*_{t-1,c} = (S_{t-1} + \Delta S_t) P'^*_{t-1,c}$. The cash after the trade is dependent on the price $C'_{t-1,c} = C_{t-1} - \Delta S_t \cdot \varphi'_{t-1,c}$. Substituting $C'_{t-1,c}$ and $p'_{t-1,c}$ in the formula for a_t, and separating out ΔS_t gives the following equation:

$$\Delta S_t = \frac{a_t C_{t-1} - S_{t-1} P'^*_{t-1,c}(1 - a_t)}{P'^*_{t-1,c}(1 - a_t) + a_t \varphi'_{t-1,c}}$$

This is close, but we still need to factor out ΔS_t from the formulae for $P'^*_{t-1,c}$ and $\varphi'_{t-1,c}$. Intuitively, the following two properties hold: (1) $a_t \geq \epsilon_{t-1,c} \iff \Delta S_t \geq 0$, and (2) $a_t \geq 0 \iff S_t \geq 0$. **TODO: Provide these proofs**.

Given these properties, we can define $P'^*_{t-1,c}$ and $\varphi'_{t-1,c}$ using variables known at decision time as follows:

$$P'^*_{t-1,c} = \begin{cases} P^b_{t-1,c} & \text{if } a_t \geq 0 \\ P^a_{t-1,c} & \text{otherwise} \end{cases}$$

$$\varphi'_{t-1,c} = \begin{cases} (1 + k) \cdot P^a_{t-1,c} & \text{if } a_t \geq \epsilon_{t-1,c} \\ (1 - k) \cdot P^b_{t-1,c} & \text{otherwise} \end{cases}$$

This completes our derivation of the trade size ΔS_t, such that $a_t = \epsilon'_{t-1,c}$ using only information available at decision-time.

C Impact of non-positive Prices and Equity

We must also consider edge cases, particularly negative equity and negative prices. Crossing zero equity causes the log-equity ratio to break down, first by being undefined, and then (when previous and current equity are both negative) by giving larger rewards when the current equity is MORE negative. PnL does not have this side-effect. On top of this the target exposure flips direction when equity is negative, making positive actions a short position and negative actions a long position. This is quite easily mitigated by normalizing by the absolute equity instead of the signed equity.

It makes sense that as equity continues to decrease that one takes on less risk, but with the target exposure, as equity continues to decrease past zero the agent starts taking on more risk again. A solution for this is to add a "lower bound" for the normalizing equity. One can view this as using a multiplicative approach as long as the equity remains above some threshold, but if it drops below you move over to an additive approach. This still allows the agent to benefit from compound interest, but comes with additional complexity.

For both the absolute and relative approaches negative price leads to the same long-short flipping. This can easily be mitigated by using the absolute price in the action formulation.

D Derivation of latency

We defined the delay as the time between timeframe $t - 1$ and the execution of the corresponding order. In order to accurately assess the duration of this delay we divide it into separate steps, starting from the moment timeframe $t - 1$ ends.

1. SERVER: Generation and publication of candle.
2. SERVER $\rightarrow$ CLIENT: Retrieval of candle
3. CLIENT: Generation of features (observation) for agent
4. CLIENT: Generation of action given features
5. CLIENT: Conversion of action to order
6. CLIENT $\rightarrow$ SERVER: Sending order to server, placing of order in queue.
7. SERVER: Execution of order

The duration of each of these steps varies greatly. **TODO: Explain reasonable latency for our scenario.**

E Position Drift

E.1 Target Positional Value

TODO: Write this section

E.2 Target Exposure

TODO: Write this section

F Cheatsheet

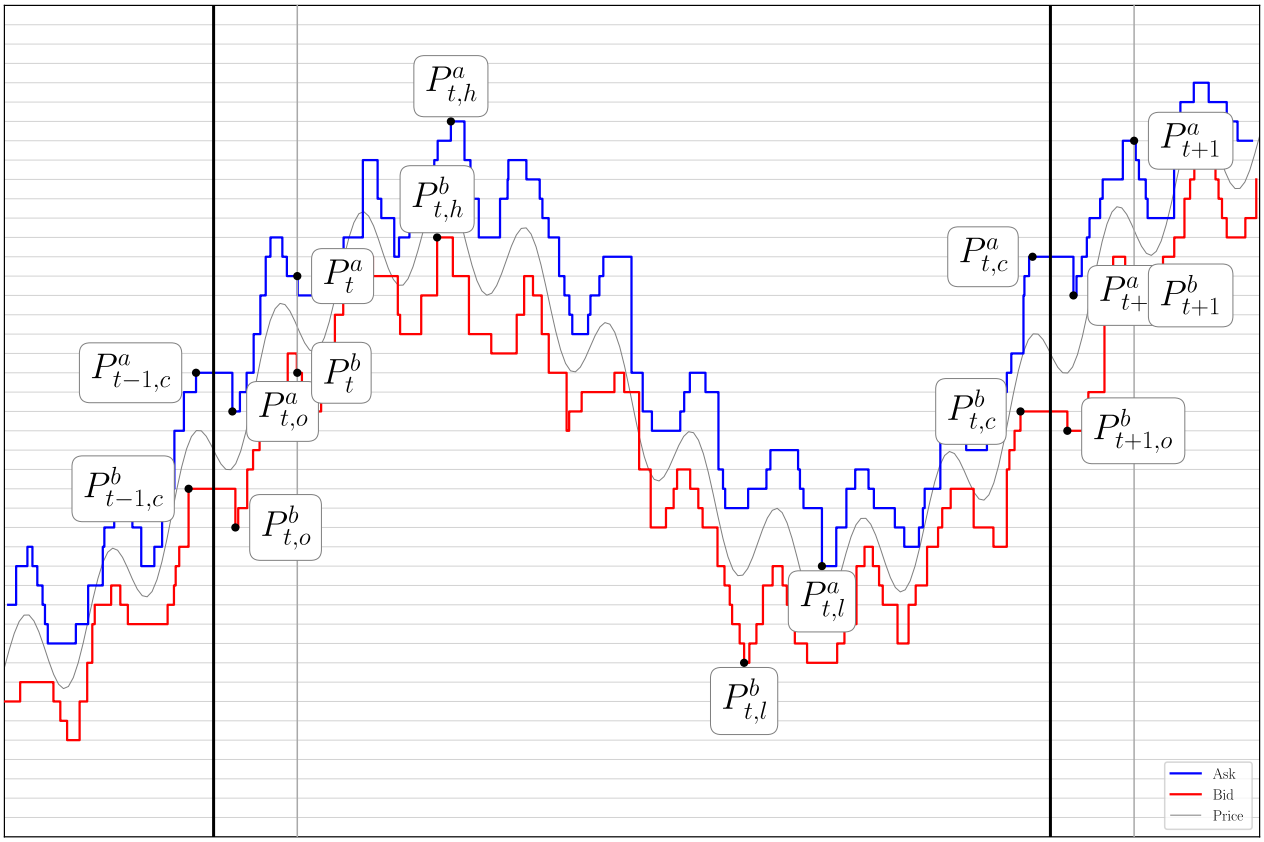

Figure 1: A diagram showing the OHLC and execution prices for some generated tick data.

Term	**Base Formula**	**Prime Variant** (X'_t)
Valuation Price	$P^*_t = \begin{cases} P^{\mathrm{b}}_t & \text{if } S_t \geq 0 \\ P^{\mathrm{a}}_t & \text{otherwise} \end{cases}$	$P'^*_t = \begin{cases} P^{\mathrm{b}}_t & \text{if } S_{t+1} \geq 0 \\ P^{\mathrm{a}}_t & \text{otherwise} \end{cases}$
Positional Value	$p_t = S_t \cdot P^*_t$	$p'_t = S_{t+1} \cdot P'^*_t$
Equity	$e_t = C_t + p_t$	$e'_t = C_{t+1} + p'_t$
Exposure	$\epsilon_t = p_t / e_t$	$\epsilon'_t = p'_t / e'_t$
Cost per share	$\varphi_t = \begin{cases} (1+k) \cdot P^{\mathrm{a}}_t & \text{if } \Delta S_t \geq 0 \\ (1-k) \cdot P^{\mathrm{b}}_t & \text{otherwise} \end{cases}$	$\varphi'_t = \begin{cases} (1+k) \cdot P^{\mathrm{a}}_t & \text{if } \Delta S_{t+1} \geq 0 \\ (1-k) \cdot P^{\mathrm{b}}_t & \text{otherwise} \end{cases}$
Updated Shares	$S_t = S_{t-1} + \Delta S_t$	$S'_t = S_t + \Delta S_{t+1}$
Updated Cash	$C_t = C_{t-1} - \Delta S_t \cdot \varphi_t$	$C'_t = C_t - \Delta S_{t+1} \cdot \varphi'_t$

Table 3: Cheat Sheet: State-Price Derivatives & Update Equations